Chapter 5 Exercises

State byte units, capacity excluding metadata, associativity, replacement, and miss-rate denominators.

CH05-Q01 Address fields

A 64-KiB data cache uses four ways, 64-byte blocks, 32-bit addresses, and LRU replacement. Data capacity excludes tags, valid/dirty bits, and replacement metadata.

1. Find blocks, sets, and tag/index/offset widths.
2. Decode `0x12345678` and `0x1a2b3c40`.
3. After installing the first address's block, does `0x12345658` hit that same block?
4. Give the first address's set and inclusive block-address range.

CH05-Q02 Associativity cost

Keep the preceding capacity, line size, and address width. Assume one tag comparator per way, with parallel comparison across the selected set.

1. Give sets and parallel tag comparators per selected set for direct, two-way, four-way, and fully associative arrangements.
2. Compute total tag bits, excluding other metadata.
3. Why can lookup slow despite unchanged data capacity?
4. Must full associativity minimize AMAT?

CH05-Q03 CPI and AMAT

In Model B all rates use retired instructions or explicitly stated data references; each penalty adds non-overlapping stall cycles not already in base CPI. Model A has one-cycle hits, 6% misses, and 80 additional cycles per miss. Independently, Model B has base CPI 1.30, 2% instruction misses, 0.30 data references/instruction, 8% data misses, and 80-cycle penalties for either miss type.

1. Find Model A's AMAT.
2. Find Model B's instruction stalls, data stalls, and total CPI.
3. Find time for 200 million instructions at 2.5 GHz.
4. Explain why full-stall addition can overestimate an out-of-order execution's waiting time.

CH05-Q04 Fill bandwidth

A four-word block contains 16 bytes. Address transfer, activations, and data transfers are serial and do not overlap; a two-word bus transfers two words per cycle. Address transfer costs one bus cycle. Each memory activation costs 15 cycles, and transferring one word on a one-word-wide bus costs one cycle. A uses one-word memory and bus. B uses two-word memory and bus. C uses two-word memory but a one-word bus.

1. Compute fill time for all three.
2. Compute effective bytes/bus cycle.
3. Explain C's benefit and remaining limitation.
4. Repeat with 40-cycle activation. Which advantage dominates?

CH05-Q05 Multiple levels

L1 has hit time one cycle and local miss rate 4%. L2 lookup costs eight cycles on every L1 miss. L2 local miss rate is 25%. An L2 miss costs another 100 cycles.

1. Find global L2 miss rate.
2. Derive AMAT.
3. Compute the erroneous answer obtained by treating 25% as a global rate.
4. What fraction of L1 misses hit L2? What happens to global misses if L1 misses fall to 2% at unchanged L2 local rate?

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH05-Q06 Page translation

Use 32-bit VA, 4-KiB pages, four-byte PTEs, and two 1024-entry page-table levels. VA `0x00403abc` misses the TLB but finds a present PTE with PPN `0x0005f`.

1. Give offset width, VPN, and offset.
2. Compute a full single-level page table's size.
3. Give both ten-bit indices.
4. Compute PA. If write permission is zero and the access is a store, identify the fault.

CH05-Q07 Dirty-page recovery

Page V is absent. Its proposed frame F currently contains dirty page W. W’ s latest contents must be retained, have no other complete copy, and receive no concurrent writes during recovery. Assume a simplified precise-exception model that permits safe retries.

1. State preservation and remapping order before overwriting F.
2. Why must a four-byte store not update architectural memory before fault recovery and authorization? Does this imply that speculative microarchitectural cache state never changes?
3. Why may PTE installation require TLB invalidation?
4. How do ASIDs help context switching, and why do they not replace permissions?

CH05-Q08 Write-back cache FSM

The four states are Idle, Compare Tag, Write-Back, Allocate. Memory has Ready 0 until completion.

1. Trace a clean read miss with exactly three Allocate cycles: Ready 0, Ready 0, Ready 1.
2. Trace a dirty write miss with one Ready cycle each for write-back and allocation.
3. When must request type and store data be saved?
4. Why must the dirty victim be preserved in memory or a safe buffer before allocation overwrites its storage? Which correctness state is lost otherwise?

CH05-Q09 MSI coherence

Use three-state write-back MSI. I has no valid copy. S permits multiple valid copies and requires current memory. M is the only valid cached copy and can leave memory stale. Both caches initially hold X in I; memory holds the old value 0. Execute P1 read X, P2 read X, P1 write X=1 after acquiring exclusivity, then an actual P2 read X causing a read miss/BusRd.

1. Trace both caches' states and memory consequences after every event.
2. Explain invalidation on the write.
3. Identify the supplier and value for the final read under basic MSI. Give both the former M owner's and requester's new states.
4. Explain why M in one cache and valid S in another is illegal. Distinguish coherence from cross-location memory consistency.

CH05-Q10 Three-C diagnosis

Both caches start empty. Case A uses a two-line direct cache with 16-byte blocks and trace `0x1000,0x1020,0x1000`. Case B uses a two-line fully associative LRU cache with the same block size and trace `0x1000,0x1040,0x1080,0x1000`.

1. Classify Case A's misses.
2. Classify Case B's misses under its explicit LRU rule.
3. Repeat Case A with the same capacity but two ways.
4. Propose a conflict reduction and state its possible hit-time or energy cost.

CH05-Q11 Dirty-victim trace and traffic

Use two-way, 64-byte blocks, four-byte stores, LRU, WB/WA. A/B/C/D map to one initially empty set; choose the lower-numbered empty way. Count payload only, no headers/acks or final flush. Access `R(A),R(B),W(A),W(C),W(D)`. Assume W(A) changes at least one byte and the cache holds its only latest copy before eviction.

1. List physical ways, dirty bits, MRU/LRU, and victim each step.
2. Count block reads, dirty write-backs, and total block transfers.
3. Restart empty under WT/no-write-allocate. Count block reads, word writes, bytes, and final set; compare.
4. Explain why dirty does not affect hit detection, and why overwriting dirty A without preserving it loses committed architectural data in this example.

CH05-Q12 VIPT, page walk, and AMAT

A system with 32-bit virtual and physical addresses has 4 KiB pages, a two-level table with ten index bits per level, and a 32 KiB, two-way L1 with 32-byte blocks. VA=0x0badf00d misses the TLB.

1. Find VPN, page offset, cache sets, index/offset widths, and PIPT tag width. Can that tag alone identify physical blocks in this VIPT configuration?
2. Does the index fit the page offset? Use VA 0/0x1000 aliases to one physical page as a counterexample. Give a safe same-capacity way count.
3. PTBR=0x00800000, PTEs four bytes. L1 PTE has PPN `0x000031`. Find both PTE addresses. L2 PTE has PPN `0x0001a`, present 1, write 0. Find PA and distinguish read/store outcomes.
4. Two PTE reads cost 25 cycles each, walk management 10 cycles, and data-cache AMAT 3 cycles. Find total miss time and its ratio to a 3-cycle TLB-hit access.

CH05-Q13 Shadow-cache classification and design tradeoffs

All caches start empty with 64 B of data and 16 B blocks. Accesses are reads only, without prefetching or coherence invalidations. The 32-bit byte-address trace is `0x0000,0x0040,0x0000,0x0010,0x0020,0x0030,0x0040,0x0000,0x0010`.

1. Convert to block numbers and give the four-line DM tag/index/offset widths. Trace DM and an equally sized FA LRU shadow, recording hit/miss and post-access state. Classify **DM's misses**.
2. How many misses does each cache have? Can subtraction give the number of conflicts? Identify the two accesses whose differences cancel.
3. Repeat with a same-capacity two-way cache, using LRU independently in each set. List both sets after every access in LRU-to-MRU order and give the hit/miss pattern.
4. DM hit time is one cycle; two-way hit time is 1.25 cycles. Each miss adds P cycles and all references are serialized. Find total times at P=20 and the range of P for which two-way is faster.

CH05-Q14 Inferring capacity and metadata

A direct-mapped write-back cache uses 32-bit byte addresses: tag bits 31–13, index bits 12–5, and offset bits 4–0. Each line also has one valid and one dirty bit. Exclude ECC and control circuitry.

1. Compute line size, line count, and data capacity.
2. Compute the bytes for tags and state bits, total storage, and the ratio of total to data capacity.
3. If write-through removes the dirty bit, all else unchanged, how many bytes are saved?

CH05-Q15 Nested translation and memory reads

Both guest and nested translations use single-level page tables, 4 KiB pages, and four-byte PTEs. The guest table starts at guest physical address (GPA) `0x2000`; the nested table starts at host physical address (HPA) `0x30000`. A read uses guest virtual address (GVA) `0x1234`. The guest PTE maps VPN `1` to guest frame `5`; nested PTEs map guest frame `2` to host frame `8` and guest frame `5` to host frame `0xa`. All mappings are valid and permit the read.

Assume no hits in a TLB, page-walk cache, or data cache. Each PTE read and final data read is a separate, serial host-memory access taking 40 cycles. The nested table's host address is known and needs no translation; ignore other overhead.

1. Find the guest PTE's GPA, the HPA of the nested PTE needed to locate it, and the guest PTE's HPA.
2. Find the data GPA, its nested PTE's HPA, and the final HPA.
3. List all host-memory reads in order and compute total cycles. If the complete translation hits in the TLB and the final data read still costs 40 cycles, what is the access time?

CH05-Q16 TLB shootdown and frame reuse

CPU 0 and CPU 1 share an address space and may both cache V→F in their TLBs. The OS on CPU 0 revokes the PTE and plans to assign frame F to a different page. Assume no DMA or pinned pages, and that prior accesses are handled under the platform's synchronization rules.

1. If the OS clears the PTE and immediately reuses F, how can CPU 1 still access the old F?
2. Order the required local invalidation, remote shootdown, completion acknowledgments, and frame reuse.
3. Can ASIDs eliminate invalidation or an equivalent mechanism for this mapping change? Explain.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 5. CSO course slides, memory hierarchy. Intel, Next Generation Microarchitecture (Nehalem), 2008, https://www.intel.com/pressroom/archive/reference/whitepaper_Nehalem.pdf

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study